

Tutorial para Iniciantes

Verificação Formal em Isabelle/HOL

Uma introdução prática às técnicas essenciais de verificação formal para novatos, cobrindo a construção de provas por indução estrutural em três domínios: números naturais, listas genéricas e árvores binárias.

Alfio Martini · 10 de dezembro de 2015

Isabelle/HOL é ao mesmo tempo um assistente de prova interativo e uma linguagem de alto nível para especificar e verificar formalmente qualquer sistema de computação que queiramos imaginar.

Visão Geral do Tutorial

Este tutorial cobre o conjunto mínimo de técnicas para que novatos possam construir provas por indução estrutural em Isabelle/HOL. Os pré-requisitos são mínimos: familiaridade com dedução natural e provas por indução estrutural.

01

Números Naturais

Definições básicas, função de adição, verificação com scripts de comandos e com a linguagem Isar.

03

Árvores Genéricas

Árvores binárias polimórficas, função de reflexão, provas sobre nodos e folhas.

02

Listas Genéricas

Tipo polimórfico de listas, função de concatenação, provas de associatividade e propriedades de reversão.

04

Comentários Finais

Metodologia recomendada: estilo procedural para exploração, Isar para documentação formal de alto nível.

O Que é uma Teoria em Isabelle?

Estrutura de uma Teoria

De forma simplificada, uma **teoria** é uma coleção de tipos, funções e teoremas — da mesma forma que um módulo ou classe em uma linguagem de programação é uma coleção de atributos, métodos e algoritmos.

Cada teoria **T** deve residir em um arquivo nomeado **T.thy**.

Formato Geral

```
theory T
imports B1 ... Bn
begin
  declarações,
  definições e
  provas
end
```

Onde **B1 ... Bn** são nomes de teorias existentes nas quais **T** é baseada. A teoria **Main** é a união de todas as teorias pré-definidas de Isabelle e deve sempre ser incluída como pai direto ou indireto.

HOL: A Lógica por Trás do Isabelle

Isabelle é um assistente de prova genérico que permite que fórmulas sejam expressas em uma linguagem formal — um tipo de lógica de segunda ordem chamada HOL. De forma simplificada:

$$\text{HOL} = \text{Programação Funcional} + \text{Lógica}$$

Estamos essencialmente estendendo a lógica de primeira ordem de forma a poder quantificar também sobre funções, conjuntos e relações. Isabelle fornece uma grande quantidade de ferramentas para provar propriedades expressas por essas fórmulas.

Verificação de Hardware e Software

Prova da correção de sistemas computacionais complexos.

Linguagens de Programação

Prova de propriedades semânticas e de tipo de linguagens.

Protocolos

Verificação formal de protocolos de comunicação e segurança.

Uma Teoria para Números Naturais

Introduzimos um tipo de dado que pode ser visto como uma cópia isomórfica do conjunto dos números naturais. Sobre os construtores deste tipo, podemos definir funções recursivas e provar várias propriedades que essas funções satisfazem.

Declaração do Tipo

```
datatype Nat = Z | suc Nat
```

A cláusula `Z` define o caso base. A cláusula `suc Nat` define o caso indutivo: ao aplicarmos o construtor `suc` a qualquer elemento do tipo `Nat`, obtemos um novo elemento do tipo `Nat`.

Os tipos dos construtores são: `Z :: Nat` e `suc :: Nat ⇒ Nat`.

Elementos do Tipo Nat

- `Z` — representa 0
- `suc Z` — representa 1
- `suc (suc Z)` — representa 2
- `suc (suc (suc Z))` — representa 3
- ... e assim por diante até `sucn Z`

 Uma vez que a definição do tipo é uma construção indutiva, é natural utilizarmos indução estrutural para verificação formal.

Regra de Indução para Nat

Toda declaração de um **datatype** em Isabelle gera automaticamente uma regra de indução. Para o tipo `Nat`, a regra é:

Regra IndNat (Meta-lógica)

Para provar uma propriedade **P** para qualquer elemento **n** do tipo `Nat`, precisamos fazer apenas duas coisas:

1. Provar que **P** vale para `Z` (caso base)
2. Assumindo que **P** vale para um elemento arbitrário `x0`, provar que **P** também vale para `suc x0` (caso indutivo)

Variáveis em Isabelle

Isabelle distingue três tipos de variáveis:

- **Variáveis livres**: ocorrem fora do escopo de qualquer quantificador
- **Variáveis ligadas**: automaticamente renomeadas para evitar conflitos
- **Variáveis esquemáticas**: prefixadas com `?`, podem ser instanciadas por qualquer termo do tipo correto durante uma prova

O quantificador universal da meta-lógica `ALL` expressa que uma variável é arbitrária, evitando condições adicionais em linguagem informal.

Função de Adição: Definição e Computação

A função de adição é definida por recursão primitiva, com indução no segundo argumento. Os rótulos `add01` e `add02` permitem referência às equações durante as provas.

```
primrec add :: Nat ⇒ Nat ⇒ Nat where
  add01: add x Z    = x
  add02: add x (suc y) = suc (add x y)
```

A função está na forma **curricada**: `add` recebe um argumento do tipo `Nat` e retorna uma função `Nat ⇒ Nat`. Por exemplo, `add (suc (suc Z))` é uma função que soma dois a qualquer natural.

Expressão Matemática	Termo em Isabelle
1 + 0	add (suc Z) Z
1 + (2 + 1)	add (suc Z) (add (suc (suc Z)) (suc Z))
(1 + 1) + 1	add (add (suc Z) (suc Z)) (suc Z)

❑ Exemplo de raciocínio equacional: `add Z (suc (suc Z)) = suc (add Z (suc Z)) [por add02] = suc (suc (add Z Z)) [por add02] = suc (suc Z) [por add01]`

Prova da Associatividade: Indução Estrutural

O teorema Th-add-01 afirma: $\forall x y z :: \text{Nat. add } x (\text{add } y z) = \text{add } (\text{add } x y) z$. Escolhemos z como variável de indução, pois a definição de `add` é recursiva no segundo argumento.

1

Definir $P\ n$

$P\ n \quad x\ y :: \text{Nat. add } x (\text{add } y n) = \text{add } (\text{add } x y) n$

A variável original z é escolhida como variável de indução por sua posição na definição recursiva.

2

Caso Base: $P\ Z$

Para x_0, y_0 arbitrários: $\text{add } x_0 (\text{add } y_0 Z) = \text{add } x_0 y_0$ [por `add01`] $= \text{add } (\text{add } x_0 y_0) Z$ [por `add01`]. q.e.d.

3

Caso Indutivo: $P\ x_0 \rightarrow P\ (\text{succ } x_0)$

Usando `add02` duas vezes e a hipótese de indução (IH): $\text{add } x_1 (\text{add } y_1 (\text{succ } x_0)) = \text{succ } (\text{add } x_1 (\text{add } y_1 x_0))$
[`add02`] $= \text{succ } (\text{add } (\text{add } x_1 y_1) x_0)$ [IH] $= \text{add } (\text{add } x_1 y_1) (\text{succ } x_0)$ [add02]. q.e.d.

Dois Estilos de Prova em Isabelle

Estilo Apply (Scripts de Comandos)

```
theorem th_add01as:  
   $\forall x y. \text{add} (\text{add } x y) z$   
    =  $\text{add } x (\text{add } y z)$   
  apply (induct_tac z)  
  apply (simp) -- caso base  
  apply (simp) -- caso indutivo  
done
```

O comando `apply (induct_tac z)` instrui Isabelle a realizar indução sobre `z`, gerando dois subobjetivos. Cada `apply (simp)` aciona o simplificador para resolver um subobjetivo. Após `done`, a variável livre `z` torna-se esquemática (`?z`).

⚠ Estilo adequado para exploração inicial, mas o texto da prova é legível apenas para a máquina — comparável a programas em linguagem assembly.

Estilo Isar (Linguagem de Alto Nível)

```
theorem th_add01isA:  
   $\forall x y. \text{add} (\text{add } x y) z$   
    =  $\text{add } x (\text{add } y z)$   
proof (induct z)  
  show  $\forall x y. \text{add} (\text{add } x y) Z$   
    =  $\text{add } x (\text{add } y Z)$   
  by simp  
next  
  fix x0::Nat  
  assume IH:  $\forall x y. \text{add} (\text{add } x y) x0$   
    =  $\text{add } x (\text{add } y x0)$   
  show  $\forall x y. \text{add} (\text{add } x y) (\text{succ } x0)$   
    =  $\text{add } x (\text{add } y (\text{succ } x0))$   
  by (simp add: IH)  
qed
```

Isar (Intelligible Semi-Automated Reasoning) reflete a notação matemática usual. Os argumentos mais importantes para este estilo são a **comunicação e manutenção**.

Prova Isar em Detalhe: Passos Atômicos

Na versão detalhada (`th_add01isB`), cada dedução é consequência da aplicação de uma única regra de simplificação, usando Isar como um **verificador de provas** (proof checker). Os comandos `also have` estabelecem o fecho transitivo implicitamente; `finally show` exporta a conclusão do bloco.

Caso Base (Passos Atômicos)

```
proof (rule allI, rule allI)
  fix x0::Nat and y0::Nat
  have add (add x0 y0) Z
    = add x0 y0
  by (simp only: add01)
  also have ...
    = add x0 (add y0 Z)
  by (simp only: add01)
  finally show
    add (add x0 y0) Z
    = add x0 (add y0 Z)
  by simp
qed
```

Caso Indutivo (Passos Atômicos)

```
fix x0::Nat and y0::Nat
have add (add x0 y0) (suc z0)
  = suc (add (add x0 y0) z0)
  by (simp only: add02)
also have ...
  = suc (add x0 (add y0 z0))
  by (simp only: IH)
also have ...
  = add x0 (suc (add y0 z0))
  by (simp only: add02)
also have ...
  = add x0 (add y0 (suc z0))
  by (simp only: add02)
finally show ... by simp
```

✓ Provas neste nível de abstração servem para verificar raciocínios passo a passo, normalmente expressos em provas construídas com lápis e papel.

Multiplicação e Pit Stop: Naturais

Função de Multiplicação

```
primrec mult :: Nat ⇒ Nat ⇒ Nat
where
  mult01: mult x Z   = Z
  mult02: mult x (suc y) =
    add x (mult x y)
```

Teoremas a provar por indução:

- **Th-mult-01:** Associatividade de `mult`
- **Th-mult-02:** Comutatividade de `mult`
- **Th-mult-03:** Distributividade de `mult` sobre `add`

Outros teoremas importantes sobre `add`: associatividade (Th-add-01), comutatividade (Th-add-02), elemento neutro à esquerda (Th-add-03), e lemas auxiliares Th-add-04 e Th-add-05.

Pit Stop: O Que Aprendemos

Até aqui, apresentamos os requisitos essenciais de Isabelle para construir provas por indução estrutural sobre funções recursivas em tipos de dados indutivos.

- O **estilo script (apply)** é muito favorável para os estágios iniciais de exploração da validade de uma conjectura e para descoberta de lemas auxiliares.
- Com **Isar**, o usuário pode construir documentos de alto nível, extremamente legíveis, que admitem a imposição de estrutura e documentação de subprovas.
- Provas em grande detalhe em Isar permitem verificar raciocínios passo a passo, como feito com lápis e papel.

Uma Teoria para Listas Genéricas

Procedemos como feito anteriormente: apresentamos a definição indutiva da nossa versão de listas genéricas, e logo após a definição recursiva da função de concatenação.

```
datatype 'a List = Empty | cons 'a ('a List)
```

List é o construtor do tipo e 'a é um parâmetro de tipo (como em ML), definindo listas **polimórficas**. Os construtores são Empty (nulo) e cons (binário: adiciona elemento à esquerda).

Tipo	Exemplos de Elementos
Nat List	Empty, cons Z Empty, cons (suc Z) Empty, ...
nat List	Empty, cons 1 Empty, cons 10 (cons 4 Empty), ...
(nat*Nat) List	Empty, cons (2, suc Z) Empty, ...

A regra de indução **IndList** gerada automaticamente requer: (1) provar $P \text{ Empty}$; (2) assumindo um elemento arbitrário $e0$ e que P vale para uma lista arbitrária $l0$, provar que P também vale para $\text{cons } e0 \text{ } l0$.

Função de Concatenação: Definição e Exemplos

Definição Recursiva

```
primrec cat :: 'a List ⇒ 'a List ⇒ 'a List
where
  cat01: cat Empty l    = l
  cat02: cat (cons h t) l =
    cons h (cat t l)
```

A indução por casos é feita no **primeiro argumento**. Os rótulos `cat01` e `cat02` permitem referência durante as provas.

Exemplo de Computação

```
cat (cons Z (cons Z Empty)) Empty
= cons Z (cat (cons Z Empty) Empty)
  [cat02, h:=Z, t:=cons Z Empty]
= cons Z (cons Z (cat Empty Empty))
  [cat02, h:=Z, t:=Empty]
= cons Z (cons Z Empty)
  [cat01, l:=Empty]
```

Tabela de Representações

Notação Matemática	Termo em Isabelle
<code>[]</code>	<code>Empty</code>
<code>[1,2,3]</code>	<code>cons 1 (cons 2 (cons 3 Empty))</code>
<code>[] @ [1+2, 0]</code>	<code>cat Empty (cons (add (suc Z) (suc (suc Z))) (cons Z Empty))</code>
<code>[1,2] @ [3,4]</code>	<code>cat (cons 1 (cons 2 Empty)) (cons 3 (cons 4 Empty))</code>

Teoremas sobre Listas

- **Th-cat-01:** Associatividade de `cat`
- **Th-cat-02:** `len (cat l1 l2) = add (len l1) (len l2)`
- **Th-cat-03:** `cat l Empty = l`
- **Th-rev-01:** `reverse (reverse l) = l`
- **Th-rev-02:** `reverse (cat l1 l2) = cat (reverse l2) (reverse l1)`

Verificação da Associatividade de Listas

Provamos o teorema $\forall l\ l2\ l3. \text{cat } l (\text{cat } l2\ l3) = \text{cat } (\text{cat } l\ l2)\ l3$ nos dois estilos. A metodologia é análoga à usada para a adição de naturais.

Estilo Apply

```
theorem th_cat01 as:  
   $\forall l\ l2\ l3.$   
     $\text{cat } l (\text{cat } l2\ l3)$   
     $= \text{cat } (\text{cat } l\ l2)\ l3$   
  apply (induct_tac l)  
  apply (simp_all)  
done
```

Utilizamos `simp_all` para simplificar todos os dois subobjetivos de uma só vez — diferença em relação à prova sobre naturais.

Estilo Isar (Versão Detalhada — Caso Indutivo)

```
fix e0::'a and l0::'a List  
assume IH:  $\forall l2\ l3.$   
   $\text{cat } l0 (\text{cat } l2\ l3)$   
   $= \text{cat } (\text{cat } l0\ l2)\ l3$   
show  $\forall l2\ l3.$   
   $\text{cat } (\text{cons } e0\ l0) (\text{cat } l2\ l3)$   
   $= \text{cat } (\text{cat } (\text{cons } e0\ l0)\ l2)\ l3$   
proof (rule allI, rule allI)  
  fix k and m  
  have  $\text{cat } (\text{cons } e0\ l0) (\text{cat } k\ m)$   
     $= \text{cons } e0 (\text{cat } l0 (\text{cat } k\ m))$   
    by (simp only: cat02)  
  also have ...  
     $= \text{cons } e0 (\text{cat } (\text{cat } l0\ k)\ m)$   
    by (simp only: IH)  
  also have ...  
     $= \text{cat } (\text{cons } e0 (\text{cat } l0\ k))\ m$   
    by (simp only: cat02)  
  also have ...  
     $= \text{cat } (\text{cat } (\text{cons } e0\ l0)\ k)\ m$   
    by (simp only: cat02)  
  finally show ... by this  
qed
```

Nota: os tipos de `e0` e `l0` não precisam ser anotados — Isabelle os infere automaticamente.

Uma Teoria para Árvores Genéricas

Procedemos como feito anteriormente: apresentamos a definição indutiva de árvores binárias genéricas e a definição recursiva da função de reflexão (espelhamento).

Declaração do Tipo

```
datatype 'a Tree =  
  Leaf  
  | Br 'a ('a Tree) ('a Tree)
```

Leaf é o construtor nulo. Br (Branch) recebe um elemento do tipo 'a e duas subárvores, retornando uma nova árvore com o elemento como raiz.

Notação	Termo
	Leaf
1, ,	Br 1 Leaf Leaf
1, 2, , , 3, ,	Br 1 (Br 2 Leaf Leaf) (Br 3 Leaf Leaf)

Regra de Indução IndTree

Para provar uma propriedade **P** para qualquer árvore **t** do tipo 'a Tree, precisamos:

1. Provar que **P** vale para Leaf
2. Assumindo um elemento arbitrário x_0 e que **P** vale para árvores arbitrárias lt_0 e rt_0 , provar que **P** também vale para $Br\ x_0\ lt_0\ rt_0$

Note que o caso indutivo agora requer **duas** hipóteses de indução (uma para cada subárvore), diferentemente dos casos anteriores.

Função de Reflexão: Definição e Teoremas

Definição Recursiva

```
primrec reflect :: 'a Tree ⇒ 'a Tree
where
  refl01: reflect Leaf = Leaf
  refl02: reflect (Br label lt rt) =
    Br label (reflect rt)
      (reflect lt)
```

A função troca recursivamente as subárvores esquerda e direita em cada nodo.

Teoremas a Provar

- **Th-refl-01:** $\forall t. \text{reflect} (\text{reflect } t) = t$
- **Th-nodes-01:** $\forall t. \text{nodesNat} (\text{reflect } t) = \text{nodesNat } t$
- **Th-nodes-02:** $\forall t. \text{leavesNat } t = \text{suc} (\text{nodesNat } t)$
- **Th-nodes-03:** $\forall t. \text{numNodes} (\text{reflect } t) = \text{numNodes } t$
- **Th-nodes-04:** $\forall t. \text{numLeaves } t = \text{numNodes } t + 1$

Prova de Th-refl-01 por Indução

Definimos $P\ t \equiv \text{reflect} (\text{reflect } t) = t$ e instanciamos:

- $P\ \text{Leaf} \quad \text{reflect} (\text{reflect } \text{Leaf}) = \text{Leaf}$
- $P\ \text{lt0} \quad \text{reflect} (\text{reflect } \text{lt0}) = \text{lt0} \text{ (IH1)}$
- $P\ \text{rt0} \quad \text{reflect} (\text{reflect } \text{rt0}) = \text{rt0} \text{ (IH2)}$
- $P\ (\text{Br } x0\ \text{lt0}\ \text{rt0}) \quad \text{reflect} (\text{reflect } (\text{Br } x0\ \text{lt0}\ \text{rt0})) = \text{Br } x0\ \text{lt0}\ \text{rt0}$

Caso Indutivo: $\text{reflect} (\text{reflect } (\text{Br } x0\ \text{lt0}\ \text{rt0})) = \text{reflect} (\text{Br } x0\ (\text{reflect } \text{rt0})\ (\text{reflect } \text{lt0})) \text{ [refl02]} = \text{Br } x0\ (\text{reflect} (\text{reflect } \text{lt0}))\ (\text{reflect} (\text{reflect } \text{rt0})) \text{ [refl02]} = \text{Br } x0\ \text{lt0}\ \text{rt0} \text{ [IH1, IH2]}. \text{ q.e.d.}$

Verificação de Árvores: Apply e Isar

Estilo Apply

```
theorem th_refl01 as:  
  reflect (reflect t) = t  
  apply (induct_tac t)  
  apply (auto)  
done
```

Utilizamos o método `auto` ao invés de `simp`. `auto` pode ser visto como uma versão sofisticada de `simp`: tenta resolver tantos subobjetivos quanto possível, utilizando o simplificador além de raciocínio lógico elementar.

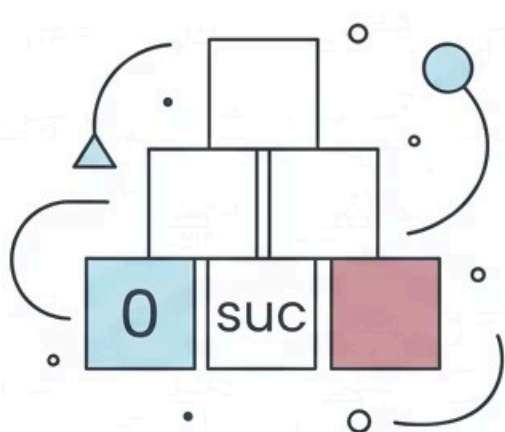
Estilo Isar (Versão Detalhada)

```
theorem th_reflect01isB:  
  reflect (reflect t) = t  
proof (induct t)  
  show reflect (reflect Leaf) = Leaf  
  proof -  
    have reflect (reflect Leaf)  
      = reflect Leaf  
    by (simp only: refl01)  
    also have reflect Leaf = Leaf  
    by (simp only: refl01)  
    finally show ?thesis by simp  
  qed  
next  
  fix x0 and lt0::'a Tree  
    and rt0::'a Tree  
  assume IH1: reflect (reflect lt0)  
    = lt0  
  assume IH2: reflect (reflect rt0)  
    = rt0  
  show reflect (reflect (Br x0 lt0 rt0))  
    = (Br x0 lt0 rt0)  
  proof -  
    have reflect (reflect (Br x0 lt0 rt0))  
      = reflect (Br x0 (reflect rt0)  
        (reflect lt0))  
    by (simp only: refl02)  
    also have ...  
      = Br x0 (reflect (reflect lt0))  
        (reflect (reflect rt0))  
    by (simp only: refl02)  
    also have ... = Br x0 lt0 rt0  
    by (simp only: IH1 IH2)  
    finally show ... by simp  
  qed  
qed
```

⚠ É necessário declarar os tipos de `lt0` e `rt0` explicitamente para que a prova tenha sucesso.

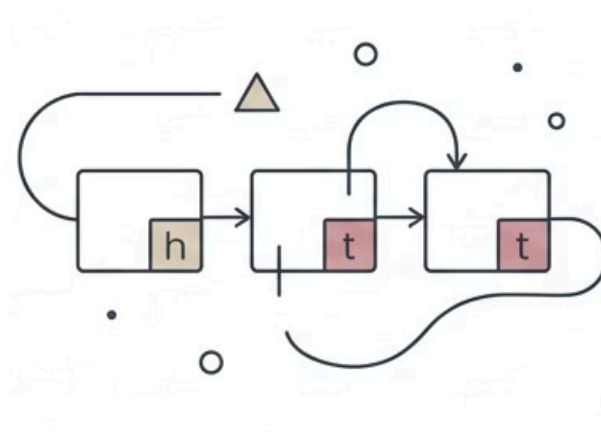
Comparando os Três Domínios

Os três capítulos do tutorial seguem a mesma metodologia, aplicada a domínios progressivamente mais complexos.



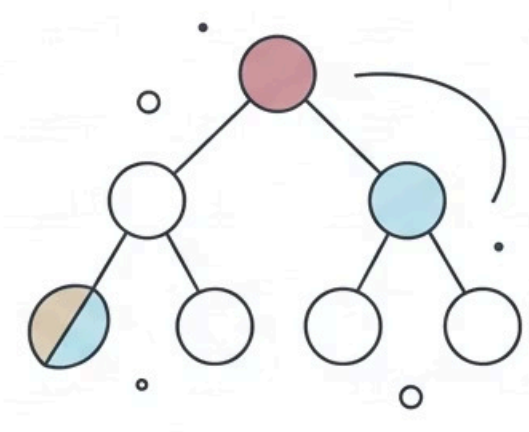
NATURAIS

- Datatype: $\text{Nat} = \mathbb{Z} \mid \text{suc Nat}$
- Operação: add
- Teoremas: asoc, com
- Indução: 2 casos



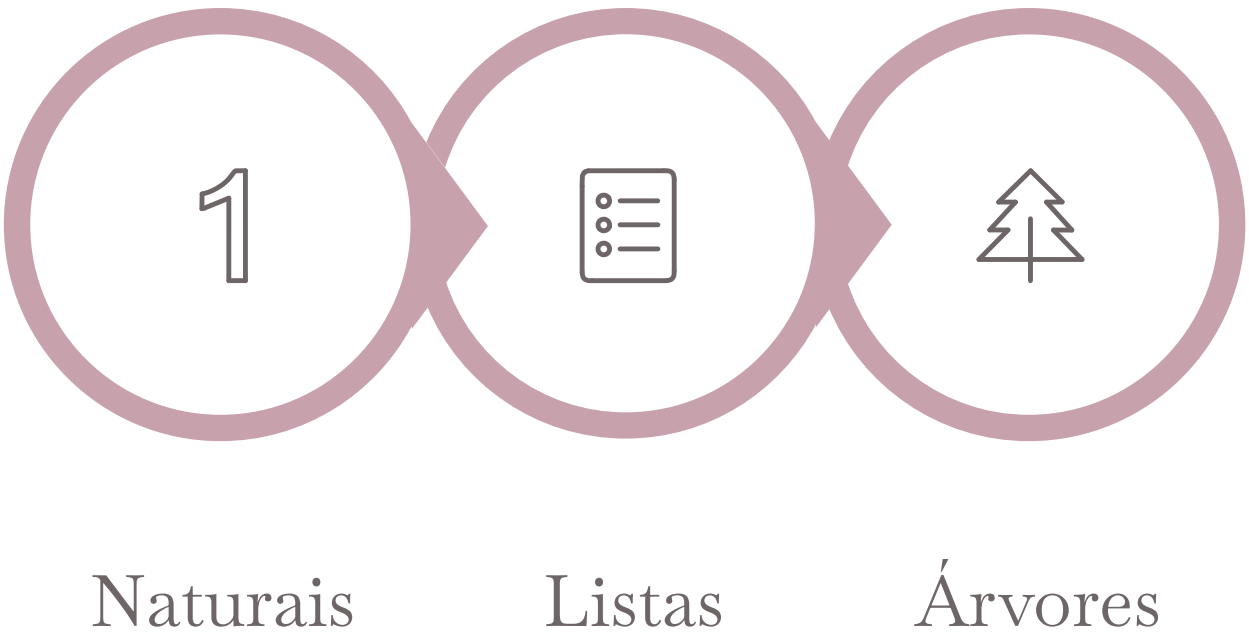
LISTAS

- Datatype: $\text{'a List} = \text{Empty} \mid \text{cons}$
- Operação: cat
- Teoremas: asoc, rev
- Indução: 2 casos



ÁRVORES

- Datatype: $\text{'a Tree} = \text{Leaf} \mid \text{Br}$
- Operação: reflect
- Teoremas: refl, nodos
- Indução: 3 casos



Metodologia Recomendada

A metodologia que usamos aconselha a utilização do estilo procedural para exploração e descoberta de possíveis lemas auxiliares. Em um segundo momento, procuramos escrever a prova na linguagem Isar, utilizando o nível de detalhe desejado.

1

Exploração

Use o estilo **apply** para explorar a validade de conjecturas e descobrir lemas auxiliares necessários.

2

Formalização

Escreva a prova em **Isar** com o nível de detalhe desejado, impondo estrutura e documentando subprovas.

3

Verificação

Para uso como **verificador de provas**, aplique passos atômicos de simplificação e utilize tantos lemas quanto necessário.

📘 Em uma próxima versão, será incorporada uma revisão de dedução natural (proof box style) e sua realização na linguagem Isabelle/Isar.

Comentários Finais

Chegamos ao final deste pequeno e breve tutorial para **newbies**. Introduzimos e listamos muitos teoremas durante as apresentações. Agora você precisa aplicar as técnicas vistas aqui e provar seus próprios teoremas.



Estilo Apply

Ideal para exploração e descoberta de lemas.
Favorável nos estágios iniciais de investigação de uma conjectura.



Linguagem Isar

Para documentos de alto nível, extremamente legíveis, com estrutura de subprovas e verificação passo a passo.



Verificação Formal

Passos atômicos de simplificação permitem verificar raciocínios como feitos com lápis e papel.

LET US NOT DREAM THAT REASON CAN EVER BE POPULAR. PASSIONS, EMOTIONS, CAN BE MADE POPULAR. BUT REASON REMAINS EVER A PROPERTY OF THE FEW.

— **Johann Wolfgang von Goethe (1749–1832)**